Rollin' Down the Biggest Hill: Analysis Draft

Correctness and running time of Dijkstra's algorithm
CS 351 Analysis of Algorithms, Project Checkpoint

The WHEEL Ones: Conor Vahey, Felicity Chin, Oscar Yaun

1. Model

A world is a directed graph $G = (V, E)$. Our test worlds are $n \times n$ grids: each cell is a node with an elevation $\mathrm{el}(v) \in \mathbb{R}$, and each cell has an edge in each direction to each of its four neighbours. The cost of an edge is

$$w(u, v) = \begin{cases} 0 & \text{if } \mathrm{el}(v) < \mathrm{el}(u) \quad \text{(downhill: roll)} \\ 1 & \text{otherwise} \quad \text{(flat or uphill: push).} \end{cases}$$

The cost of a path is its number of pushes. $\delta(s, t)$ is the minimum cost of any path from s to t. Every edge has a reverse edge, so every node can reach every other node. On a grid $|E| = 4n(n-1) < 4|V|$. All costs are non-negative, which is the property Dijkstra's algorithm needs.

We use the *subpath property*: if P is a shortest path from s to u and y lies on P, then the part of P from s to y is a shortest path from s to y, since otherwise swapping in a cheaper path would make P cheaper [1, Lemma 22.1].

2. The algorithm as implemented

(`heely/dijkstra.py`.) Each node v has a tentative distance $d[v]$, initially ∞ except $d[s] = 0$, and a flag *settled*, initially false. A binary min-heap holds pairs (d, v). Repeat until the heap is empty: pop the pair with the smallest d; if v is already settled, skip it; otherwise mark v settled, and stop if $v = t$; otherwise for each edge (v, x), if $d[v] + w(v, x) < d[x]$, set $d[x] \leftarrow d[v] + w(v, x)$, set $\mathrm{parent}[x] \leftarrow v$, and push $(d[x], x)$. Instead of decreasing a key in the heap we push a new pair and skip old ones when they come out (*lazy deletion*), which is the standard way to use Python's `heapq`. Otherwise this is the algorithm in CLRS Section 22.3 [1].

3. Correctness

Lemma 1. *When a pair (d, v) is popped and v is not yet settled, $d = d[v]$, and $d[v] \le d[x]$ for every unsettled node x.*

Proof. Let (d, v) be an arbitrary popped pair with v unsettled. Every time $d[v]$ was lowered, a pair with the new value was pushed, so the heap holds $(d[v], v)$, and $d[v] \le d$ because $d[v]$ only decreases. Since the heap pops the smallest key and (d, v) came out, $d \le d[v]$, so $d = d[v]$. In the same way every unsettled x with $d[x] < \infty$ has a pair $(d[x], x)$ in the heap, and the heap returned the smallest, so $d[v] \le d[x]$. Because (d, v) was arbitrary, the lemma holds for every pop. □

Theorem 2 (CLRS Theorem 22.6)**.** *When a node u is settled, $d[u] = \delta(s, u)$. In particular, when the algorithm stops at t, $d[t] = \delta(s, t)$, and following parent pointers from t back to s gives a path of cost $\delta(s, t)$.*

Proof. First, $d[u] \ge \delta(s, u)$ always, because $d[u]$ is either ∞ or the cost of a real path built by relaxations, and no path costs less than the distance. Now suppose, for contradiction, that some node is settled with d larger than its distance, and let u be the first such node. Then $u \ne s$, since

$d[s] = 0 = \delta(s, s)$. Let P be a shortest path from s to u. When u is settled, s is settled and u is not, so let y be the first node on P that is not settled, and x the node just before it, which is settled. Since u is the first bad node, $d[x] = \delta(s, x)$. When x was settled the edge (x, y) was relaxed, so $d[y] \leq \delta(s, x) + w(x, y) = \delta(s, y)$ by the subpath property, and together with $d[y] \geq \delta(s, y)$ this gives $d[y] = \delta(s, y)$. Since y is on P no later than u and costs are non-negative, $\delta(s, y) \leq \delta(s, u)$. So $d[y] \leq \delta(s, u) < d[u]$. But u was popped while y was unsettled, so by Lemma 1 $d[u] \leq d[y]$, a contradiction. So no such u exists.

For the path: when parent$[x]$ is set to v, v is settled, so $d[v]$ is final and $d[x] = d[v] + w(v, x)$. Once x is settled, $d[x]$ and parent$[x]$ never change. So walking parents back from t passes through settled nodes, each settled strictly earlier than the one before, so the walk cannot loop, and it can only end at s, the one settled node without a parent. Each step lowers d by exactly the edge cost, so the path costs $d[t] - d[s] = \delta(s, t)$. □

4. Running time

Theorem 3. *Dijkstra's algorithm with a binary heap and lazy deletion runs in $O((|V| + |E|) \log |V|)$ time and $O(|V| + |E|)$ space. On an $n \times n$ grid this is $O(n^2 \log n)$.*

Proof. Each node is settled at most once, and a node's outgoing edges are relaxed only when it is settled, so there are at most $|E|$ relaxations. That means at most $|E|+1$ pushes and at most $|E|+1$ pops. The heap never holds more than $|E|+1$ pairs, so each push or pop costs $O(\log |E|) = O(\log |V|)$, since $|E| \leq |V|^2$. Setting up the arrays costs $O(|V|)$. The total is $O(|V|+|E| \log |V|) = O((|V|+|E|) \log |V|)$. The arrays take $O(|V|)$ space, the adjacency lists $O(|V| + |E|)$, and the heap $O(|E|)$. On a grid $|E| < 4|V| = 4n^2$, so the time is $O(n^2 \log n)$. □

Stopping when t is settled does not change the worst case, but in practice it means only the nodes closer to s than t (plus some ties) are settled. The preliminary results show about half of all nodes being settled on average, which is what the second algorithm, ALT, aims to reduce.

Sources

References

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. Chapter 22, Lemma 22.1 and Theorem 22.6.